

Primary sources & further reading

Track 16.7 —  Claude Code and the Agent SDK · 16-ai-tech-stack/tracks/claude-code/papers/PAPERS.md · 16-ai-tech-stack/tracks/claude-code/resources/RESOURCES.md

Papers & Primary Sources — Claude Code & the Agent SDK

Claude Code is a product, not a paper, so "primary sources" here means the engineering write-ups that explain *why* it behaves the way it does, plus the research the design descends from. Read the docs alongside these, not instead of them — and re-check the docs before you argue with either.

Start here (Anthropic engineering, in this order)

#	Source	Why it matters	Link
1	Building Effective Agents — Anthropic Engineering	The workflows-vs-agents distinction this track leans on. Defines when a harness should decide vs. when the model should.	https://www.anthropic.com/engineering/building-effective-agents
2	Claude Code: Best Practices for Agentic Coding — Anthropic	The official usage write-up: CLAUDE.md, permissions, planning mode, subagents, headless runs.	https://www.anthropic.com/engineering/claude-code-best-practices
3	Effective Context Engineering for AI Agents — Anthropic	Why compaction, note-taking and subagent isolation exist: attention budget, not just token count.	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
4	Writing Effective Tools for Agents — Anthropic	Tool design as interface design. Pairs with the SDK custom-tools section of the notes.	https://www.anthropic.com/engineering/writing-tools-for-agents

The research underneath

Paper	Year	One-line takeaway	Link
ReAct: Synergizing Reasoning and Acting in Language Models — Yao et al.	2022	The think-act-observe loop the harness implements; worth reading to see how much is now plumbing	arXiv:2210.03629
SWE-bench: Can Language Models Resolve Real-World GitHub Issues? — Jimenez et al.	2023	The benchmark that made "agentic coding" measurable, and its contamination caveats	arXiv:2310.06770
Design Patterns for Securing LLM Agents from Prompt Injection Attacks — Beurer-Kellner et al.	2025	The Action-Selector / Plan-Then-Execute / Dual-LLM patterns; the academic version of the permission story	arXiv:2506.08837
Not with my name! / InjecAgent — tool-integration attack studies	2024	Direct task-oriented attacks on tool-using agents; the threat model for hooks and deny rules	arXiv:2403.02692

The specification that matters

Source	Why	Link
Model Context Protocol — spec + docs	The open standard the tool ecosystem speaks; the SDK's custom tools are an in-process MCP server	https://modelcontextprotocol.io/

The blog post every practitioner should read once

- **The Lethal Trifecta** — Simon Willison (2025). Private data + untrusted content + outward communication in one agent = prompt injection you cannot prompt your way out of. It is the one-URL threat model briefing for everything in the permissions sections:
<https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>

How to read a product changelog like a paper

Agent harnesses ship weekly, and the interesting signal is not the feature list — it is *what got moved from "model judgement" to "harness enforcement"* (or back). Read Claude Code's changelog with that question in mind: <https://code.claude.com/docs/en/changelog>

Resources — Claude Code & the Agent SDK

Official documentation (start here, keep open)

Page	What's inside
https://code.claude.com/docs	The front door; the quickstart is genuinely quick
https://code.claude.com/docs/en/memory	CLAUDE.md, rules, auto memory — the "project memory" surface
https://code.claude.com/docs/en/skills	Skills: model-invoked capabilities and reference material
https://code.claude.com/docs/en/sub-agents	Subagents: separate context windows, separate prompts
https://code.claude.com/docs/en/hooks	Hook events (PreToolUse/PostToolUse/...), config, exit-code semantics
https://code.claude.com/docs/en/settings	settings.json, permission modes, allow/ask/deny rule syntax
https://code.claude.com/docs/en/mcp	MCP server configuration in Claude Code
https://code.claude.com/docs/en/changelog	Ships weekly; read before trusting anything above
https://docs.claude.com/en/docs/claude-code/sdk	Agent SDK overview, then Python / TypeScript subpages

Repositories worth cloning

Repo	What's inside
https://github.com/anthropics/claude-code	The public repo: issues, examples, the release notes people actually read
https://github.com/anthropics/claude-agent-sdk-python	Python SDK: query loop, tools, hooks, permission callbacks
https://github.com/anthropics/claude-agent-sdk-typescript	TypeScript SDK, same loop, different ergonomics
https://github.com/anthropics/skills	Anthropic's own skills — the reference style for writing yours
https://github.com/anthropics/anthropic-cookbook	Agent patterns and tool-use recipes beyond Claude Code itself
https://github.com/modelcontextprotocol/servers	Reference MCP servers: filesystem, git, fetch — read one before writing your own

Community & keeping current

- [r/ClaudeAI](#) — the largest working-practices forum; filter hard for signal vs. vibes
- The changelog RSS discipline: skim weekly, adopt one new mechanism per month, not ten
- Simon Willison's blog — the best running commentary on agent safety in practice:
<https://simonwillison.net/>

Also in this repo

- The offline SDK reference script: `../code/agent_sdk_quickstart.py`
- The guardrail hook pair to install and attack: `../code/example-hooks-settings.json`
- See also `../13-ai-security/` for the injection deep dive and `../09-mcp/` for the protocol itself.